# CS 216: Introduction to Blockchain

## From Command Line to Programming: A Hands-on Approach

**Team Members**

| | |
|---|---|
| Arnav Kumar | 240001013 |
| Aryaman Awanish Tiwari | 240001014 |
| Ayush Singh Rana | 240001015 |
| Hrishabh Mittal | 240001035 |


**GitHub Repository:**
`https://github.com/Stonky-Boi/Bitcoin-Transaction-Lab-Assignment`

## Contents

**1 Part 1: Legacy Address Transactions (P2PKH) 1**
1.1 System Architecture & RPC Client . . . 1
1.2 Workflow . . . 2
1.3 Decoded Scripts . . . 2
1.4 Script Structure and Validation . . . 2
1.5 Execution Analysis . . . 3
1.6 Bitcoin Debugger Analysis . . . 3

**2 Part 2: P2SH-SegWit Address Transactions 5**
2.1 Workflow . . . 5
2.2 Decoded Scripts . . . 5
2.3 Script Structure and Validation . . . 5
2.4 Execution Analysis . . . 6
2.5 Bitcoin Debugger Analysis . . . 6

**3 Part 3: Comparative Analysis 10**
3.1 Transaction Size Data . . . 10
3.2 Explanation of SegWit Benefits . . . 10

# 1 Part 1: Legacy Address Transactions (P2PKH)

## 1.1 System Architecture & RPC Client

To fulfill the programming requirements and automate the transaction workflows, a custom C++ RPC client (`BitcoinRpcClient`) was engineered to interface directly with the local `bitcoind` node running in `regtest` mode via HTTP POST requests. This utility systematically handles wallet operations, proper UTXO handling, and the precise creation, signing, and broadcasting of both transaction chains. Crucially, this programmatic foundation successfully utilizes the `decoderawtransaction` RPC call to automatically extract the underlying locking and unlocking scripts for further verification and detailed analysis in `btcdeb`.

## 1.2 Workflow

This section outlines the transaction workflow for the Legacy (P2PKH) addresses. First, the local `bitcoind` node was initialized in `regtest` mode and 101 blocks were mined to a mining address to mature the coinbase funds. Three legacy addresses were generated: Address A, Address B, and Address C. Address A was funded with 2.0 BTC from the coinbase.

Using the RPC client, a raw transaction was created to send 1.999 BTC from Address A to Address B. After signing and broadcasting this transaction, the resulting TXID was recorded. Subsequently, the UTXO associated with this TXID (where Address B is the recipient) was fetched using `listunspent` and utilized as the input for a second transaction, which sent 1.998 BTC from Address B to Address C.

## 1.3 Decoded Scripts

The following decoded scripts were extracted using the `decoderawtransaction` RPC call for the Legacy transactions:

**Challenge Script (Locking Script / ScriptPubKey for Address B):**

```
OP_DUP OP_HASH160 <PubkeyHash_of_B> OP_EQUALVERIFY OP_CHECKSIG
```

**Response Script (Unlocking Script / ScriptSig from Address B spending to C):**

```
<Signature_of_B> <PublicKey_of_B>
```

## 1.4 Script Structure and Validation

The validation of a Legacy P2PKH transaction works by concatenating the input's `scriptSig` (response) with the referenced output's `scriptPubKey` (challenge). The concatenated script runs on a stack-based execution model.

1. The `scriptSig` pushes the `<Signature>` and `<PublicKey>` of Address B onto the stack.
2. The `scriptPubKey` begins executing. `OP_DUP` duplicates the top item (`<PublicKey>`).
3. `OP_HASH160` hashes the duplicated public key to produce a PubkeyHash.
4. The expected `<PubkeyHash_of_B>` is pushed onto the stack.
5. `OP_EQUALVERIFY` checks if the calculated hash matches the expected hash and consumes both; if they do not match, execution fails.
6. Finally, `OP_CHECKSIG` consumes the `<Signature>` and the original `<PublicKey>` to verify the cryptographic signature against the transaction data. If valid, it pushes `TRUE` to the stack, allowing the transaction to pass validation.

## 1.5 Execution Analysis

```
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$ ./build/legacyTxs.out

=== Setting Up Environment ===
Mining 101 blocks to mature coinbase funds...

=== Legacy Addresses (P2PKH) ===
Address A: movJaEnXcEbkK4YQ1XMEfJWZZ8GrLj9j33
Address B: n43a22JseqgqUA1YXHbNHKRrHjvNah2B3y
Address C: n1JbXrhYnUdzNQkAWYjtvGMx9yorqE6iwW

Funding Address A with 2.0 BTC...

=== Transaction A to B ===
Locking Script (ScriptPubKey) for B:
OP_DUP OP_HASH160 f71d1110e831a75c8163f87d30f6813afc1e4f37 OP_EQUALVERIFY OP_CHECKSIG

Broadcasted TX A->B with TXID: 867960f5a8606a9ac156185c9c07e9a06af50989a38b0a116046445261cc8720

=== Transaction B to C ===
Unlocking Script (ScriptSig) from B:
304402202ea4d77b8b7766d02c0617a25cd5883585a3509e6ae93f532bf691b15730791f022058c6588dcc004307bb418f3262ea482d2f506d22604c
f8a4e80aa732f6221641[ALL] 021199ebbc2360e568f13cd3094ebd0aba5e3090bee17dd38bc6a8eb46221d647d

Broadcasted TX B->C with TXID: 1426e406e39e964f3de62f258defe7ea060e719dd008f3433c87b8c087735b82
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$
```

Figure 1: Terminal execution of the P2PKH workflow (`legacyTxs.out`).

## 1.6 Bitcoin Debugger Analysis

The sequence below illustrates the `btcdeb` execution of our extracted P2PKH challenge and response scripts. The debugger traces the hashing of the public key, the equality verification against the expected PubkeyHash, and the final signature check.

```
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$ btcdeb '[OP_DUP OP_HASH160 aa7118476a97f921c9a4cac95915196d1f340318 OP
_EQUALVERIFY OP_CHECKSIG]' 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575a2cf7ae2402207b6b9cda4238950d7
49f5e756d086a7c62a225938093809f650e7dacffe9c82501 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
btcdeb 0.3.20 -- type `btcdeb -h` for start up options
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
5 op script loaded. type `help` for usage information
script                                    |                                                                stack
------------------------------------------+----------------------------------------------------------------
OP_DUP                                    | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
OP_HASH160                                | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
aa7118476a97f921c9a4cac95915196d1f340318 |
OP_EQUALVERIFY                            |
OP_CHECKSIG                               |
#0000 OP_DUP
btcdeb>
```

(a) Initial Stack State

Figure 2: Step-by-step execution of the Legacy P2PKH scripts using `btcdeb`.

```
btcdeb> step
                <> PUSH stack 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
script                                  |                                                                stack
----------------------------------------+----------------------------------------------------------------
OP_HASH160                              | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
aa7118476a97f921c9a4cac95915196d1f340318 | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
OP_EQUALVERIFY                          | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
OP_CHECKSIG                             |
#0001 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack aa7118476a97f921c9a4cac95915196d1f340318
script                                  |                                                                stack
----------------------------------------+----------------------------------------------------------------
aa7118476a97f921c9a4cac95915196d1f340318 |                          aa7118476a97f921c9a4cac95915196d1f340318
OP_EQUALVERIFY                          | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
OP_CHECKSIG                             | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
#0002 aa7118476a97f921c9a4cac95915196d1f340318
btcdeb> step
                <> PUSH stack aa7118476a97f921c9a4cac95915196d1f340318
script                                  |                                                                stack
----------------------------------------+----------------------------------------------------------------
OP_EQUALVERIFY                          |                          aa7118476a97f921c9a4cac95915196d1f340318
OP_CHECKSIG                             |                          aa7118476a97f921c9a4cac95915196d1f340318
                                        | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
                                        | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
#0003 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                                stack
----------------------------------------+----------------------------------------------------------------
OP_CHECKSIG                             | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
                                        | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
#0004 OP_CHECKSIG
btcdeb>
```

(b) Hashing & Equality Verification

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                                stack
----------------------------------------+----------------------------------------------------------------
OP_CHECKSIG                             | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
                                        | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
#0004 OP_CHECKSIG
btcdeb> step
error: Signature must be zero for failed CHECK(MULTI)SIG operation
btcdeb> step
script                                  |                                                                stack
----------------------------------------+----------------------------------------------------------------
                                        | 03f1d4c6393316fe3f6ef36014533c5243ff278668f6141c09fb3df65fb5c9173e
                                        | 304402201e003485fea7a639274bab917525796217ded0715ca30ea93e29575...
#0004 OP_CHECKSIG
btcdeb> step
at end of script
btcdeb>
```

(c) Final Step (OP_CHECKSIG)

# 2 Part 2: P2SH-SegWit Address Transactions

## 2.1 Workflow

This workflow mirrors Part 1 but utilizes the `p2sh-segwit` address type. Addresses A', B', and C' were generated. After funding Address A' with 2.0 BTC, a transaction was constructed to send 1.999 BTC from A' to B'. The TXID from this broadcast was queried to locate the UTXO belonging to B'. This UTXO was then referenced as the input to create and sign a final transaction sending 1.998 BTC from B' to C'.

## 2.2 Decoded Scripts

The `decoderawtransaction` output revealed the following script structures for the P2SH-P2WPKH transactions:

**Challenge Script (ScriptPubKey for Address B'):**

```
OP_HASH160 <Hash160(RedeemScript)> OP_EQUAL
```

**Response Script (ScriptSig from Address B' spending to C'):**

```
<RedeemScript> (Format: 0014<20-byte-PubkeyHash>)
```

## 2.3 Script Structure and Validation

P2SH allows the sender to commit to a complex script by only providing its hash. For P2SH-P2WPKH, the SegWit logic is embedded inside a familiar P2SH address to ensure backward compatibility.

When validating, the legacy nodes only see the `scriptSig` (which only contains the Redeem Script) and the `scriptPubKey`.

1. The `scriptSig` pushes the `<RedeemScript>` onto the stack.
2. The `scriptPubKey` hashes this script using `OP_HASH160` and compares it to the committed hash using `OP_EQUAL`.
3. If this succeeds, upgraded SegWit nodes detect that the Redeem Script is a valid SegWit program (version 0, followed by a 20-byte hash).
4. The node then evaluates the SegWit program by taking the `<Signature>` and `<PublicKey>` from the separate Witness field and validating them against the 20-byte PubkeyHash inside the Redeem Script, entirely independent of the legacy script execution.

## 2.4 Execution Analysis

```
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$ ./build/segWitTxs.out

=== P2SH-SegWit Addresses ===
Address A': 2N82bZsCYhKtLS9fbPqax5U4CEZLdcqhrKt
Address B': 2MvP6Q7VvPXgZ4FMBTyJWkWPfbseYdHSx2G
Address C': 2N14YA3qU2hVtEtAf4YXQEaxS7avjrqxG1z

Funding Address A' with 2.0 BTC...

=== Transaction A' to B' ===
Locking Script (ScriptPubKey Challenge) for B':
OP_HASH160 22655eb42af78c663d31dd6c8347d5894b0d893b OP_EQUAL

Broadcasted TX A'->B' with TXID: 44e559ab214a69d17d29f32c4cc002577979e487838ce6c879645817b3402f8a

=== Transaction B' to C' ===
Redeem Script (ScriptSig) from B':
00141cda335663c682a01821452354e273e252132125

Witness Data (Signature & Public Key):
304402207117ba0c4ae6e11ae5fd25e6495d3e4d42bbbf913402b4568d67395191e96ded02205bba7f45e491f0a81740cb2c20e9c21b25b3780f3f30
63e3fa0cd9a4839ea83c01
03298b238b8a7eed310f5f11626afc87338d8d83a850de9dad6795c59ddeefa99a

Broadcasted TX B'->C' with TXID: a69f94a7900f3efdd04eac862045a767ed5ab297f92b309c9da80a9abb01e641
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$
```

Figure 3: Terminal execution of the P2SH-SegWit workflow (`segWitTxs.out`).

## 2.5 Bitcoin Debugger Analysis

Debugging a P2SH-P2WPKH transaction requires a two-phase process. First, the outer P2SH wrapper is validated. Once `OP_EQUAL` passes, the inner SegWit witness program is independently extracted and validated using the witness stack data.

```
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$ btcdeb '[OP_HASH160 2a4eec6eb5a9967b868
' 001415f25a6669d0103ca10ed6cb154f2dda538407e3
btcdeb 0.3.20 -- type `btcdeb -h` for start up options
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
6 op script loaded. type `help` for usage information
script                                  |                                        stack
----------------------------------------+-----------------------------------------
OP_HASH160                              | 001415f25a6669d0103ca10ed6cb154f2dda538407e3
2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b |
OP_EQUAL                                |
<<< P2SH script >>>                     |
0                                       |
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
#0000 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b
script                                  |                                        stack
----------------------------------------+-----------------------------------------
2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b |     2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b
OP_EQUAL                                |
<<< P2SH script >>>                     |
0                                       |
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
#0001 2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b
btcdeb> step
                <> PUSH stack 2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b
script                                  |                                        stack
----------------------------------------+-----------------------------------------
OP_EQUAL                                |     2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b
<<< P2SH script >>>                     |     2a4eec6eb5a9967b8688acdcd2f9c14b8df9118b
0                                       |
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
#0002 OP_EQUAL
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                        stack
----------------------------------------+-----------------------------------------
<<< P2SH script >>>                     |                                           01
0                                       |
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
<<< P2SH script >>>
```

(a) Phase 1: P2SH Wrapper Loaded

Figure 4: Two-phase chronological execution of the P2SH-P2WPKH and Witness scripts using `btcdeb`.

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                       stack
----------------------------------------+-------------------------------------------
<<< P2SH script >>>                     |                                          01
0                                       |
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
<<< P2SH script >>>
btcdeb> step
                <> POP  stack
script                                  |                                       stack
----------------------------------------+-------------------------------------------
0                                       |
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
#0004 0
btcdeb> step
                <> PUSH stack
script                                  |                                       stack
----------------------------------------+-------------------------------------------
15f25a6669d0103ca10ed6cb154f2dda538407e3 |                                          0x
#0005 15f25a6669d0103ca10ed6cb154f2dda538407e3
btcdeb> step
                <> PUSH stack 15f25a6669d0103ca10ed6cb154f2dda538407e3
script                                  |                                       stack
----------------------------------------+-------------------------------------------
                                        |     15f25a6669d0103ca10ed6cb154f2dda538407e3
                                        |                                          0x
btcdeb> step
script                                  |                                       stack
----------------------------------------+-------------------------------------------
                                        |     15f25a6669d0103ca10ed6cb154f2dda538407e3
                                        |                                          0x
btcdeb> step
at end of script
btcdeb>
```

(b) Phase 1: P2SH Wrapper Conclusion

```
0aa3d14afb85282d16353d5bb8462c5682b5a4c2e13466801 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
btcdeb 0.3.20 -- type `btcdeb -h` for start up options
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
5 op script loaded. type `help` for usage information
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_DUP                                  | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
OP_HASH160                              | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
15f25a6669d0103ca10ed6cb154f2dda538407e3 |
OP_EQUALVERIFY                          |
OP_CHECKSIG                             |
#0000 OP_DUP
btcdeb> step
                <> PUSH stack 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_HASH160                              | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
15f25a6669d0103ca10ed6cb154f2dda538407e3 | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
OP_EQUALVERIFY                          | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
OP_CHECKSIG                             |
#0001 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 15f25a6669d0103ca10ed6cb154f2dda538407e3
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
15f25a6669d0103ca10ed6cb154f2dda538407e3 |                                  15f25a6669d0103ca10ed6cb154f2dda538407e3
OP_EQUALVERIFY                          | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
OP_CHECKSIG                             | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
#0002 15f25a6669d0103ca10ed6cb154f2dda538407e3
btcdeb> step
                <> PUSH stack 15f25a6669d0103ca10ed6cb154f2dda538407e3
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_EQUALVERIFY                          |                                  15f25a6669d0103ca10ed6cb154f2dda538407e3
OP_CHECKSIG                             |                                  15f25a6669d0103ca10ed6cb154f2dda538407e3
                                        | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
                                        | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
#0003 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_CHECKSIG                             | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
                                        | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
#0004 OP_CHECKSIG
btcdeb>
```

(c) Phase 2: Inner Witness Program

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_CHECKSIG                             | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
                                        | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
#0004 OP_CHECKSIG
btcdeb> step
error: Signature must be zero for failed CHECK(MULTI)SIG operation
btcdeb> step
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
                                        | 0365142b9ed6f725fdebd9b76d1b6fe097d17b821a089552cb83549b3c3c2d0222
                                        | 304402200695fce21a009eb65c8586e3c4564a218bcb326a18dc436d0fa0cef...
#0004 OP_CHECKSIG
btcdeb> step
at end of script
btcdeb>
```

(d) Phase 2: Witness Conclusion

# 3 Part 3: Comparative Analysis

## 3.1 Transaction Size Data

A dynamic C++ analysis program was written to extract and tabulate the exact dimensions of both transaction formats based on our local `regtest` output.

```
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$ ./build/comparison.out

=== Transaction Size Analysis ===

Format                  Transaction    Size(Bytes)    VSize(vBytes)  Weight(WU)
--------------------------------------------------------------------------------
Legacy (P2PKH)          A->B           191            191            764
Legacy (P2PKH)          B->C           191            191            764
SegWit (P2SH-P2WPKH)    A'->B'         215            134            533
SegWit (P2SH-P2WPKH)    B'->C'         215            134            533
[ASR@Omen-16 Bitcoin-Transaction-Lab-Assignment]$
```

Figure 5: Output of the C++ comparison utility, reading directly from the generated CSV metrics.

| Format | Transaction | Size (Bytes) | VSize (vBytes) | Weight (WU) |
|---|---|---|---|---|
| Legacy (P2PKH) | A → B | 191 | 191 | 764 |
| Legacy (P2PKH) | B → C | 191 | 191 | 764 |
| SegWit (P2SH-P2WPKH) | A' → B' | 215 | 134 | 533 |
| SegWit (P2SH-P2WPKH) | B' → C' | 215 | 134 | 533 |

Table 1: Data-driven comparison of Transaction Sizes and Weights generated via the RPC integration.

## 3.2 Explanation of SegWit Benefits

As demonstrated in the table, while the absolute size in raw Bytes is slightly larger for P2SH-P2WPKH (215 bytes vs 191 bytes) due to the P2SH wrapper overhead, the **VSize (vBytes)** and **Weight** are significantly smaller (134 vBytes vs 191 vBytes).

This reduction occurs because Segregated Witness (SegWit) separates the signature (witness) data from the core transaction structure. In the Bitcoin Weight measurement system, traditional transaction bytes are multiplied by 4, but witness bytes are only multiplied by 1. Consequently, moving the heavy cryptographic signatures out of the `scriptSig` and into the witness section heavily discounts their weight cost.

The primary benefit of this architecture is that it reduces the size counted against block limits for SegWit transactions, lowering network fees. Because miners calculate transaction fees based on vBytes rather than raw bytes, users pay roughly 30% lower fees when utilizing SegWit addresses compared to standard Legacy addresses for a basic 1-input, 1-output transaction.